Wickham's Layered Grammar & ggplot2

Workshop 2 — Module 2

Prof.Asoc. Endri Raco

2025/26

Data Visualization for Data Scientists
UNYT Tirana

Module Roadmap

1. The ggplot2 template: the pattern behind every plot
2. `aes()`: mapping vs setting
3. The geom catalogue and default statistics
4. Position adjustments: dodge, stack, fill, jitter
5. Aesthetic inheritance: global vs local
6. Complete worked example in R and Python

Learning Outcome
You will master the ggplot2 syntax and construct any chart by composing layers with +.

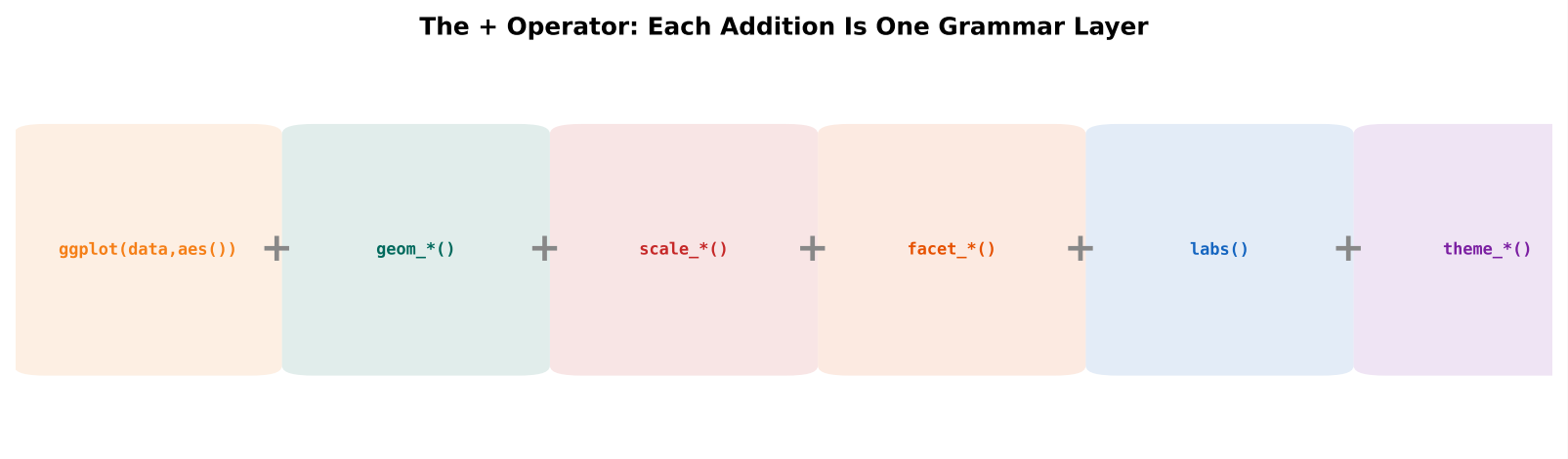
The ggplot2 Template

The Pattern Behind Every Plot

The ggplot2 Template: Every Plot Follows This Pattern

```
ggplot(data,                                          <- Start: bind data frame
       aes(x = var1, y = var2,                        <- Map variables to channels
           color = var3)) +
  geom_point(size=2, alpha=0.6) +                     <- Layer 1: geometry + params
  geom_smooth(method='lm', se=TRUE) +                 <- Layer 2: stat + geometry
  scale_color_brewer(palette='Set2') +                <- Scale: data -> visual property
  facet_wrap(~var4, ncol=3) +                         <- Facets: split into panels
  labs(title='...', x='...') +                        <- Labels: title, axes, legend
  theme_minimal(base_size=11)                         <- Theme: all non-data elements
```

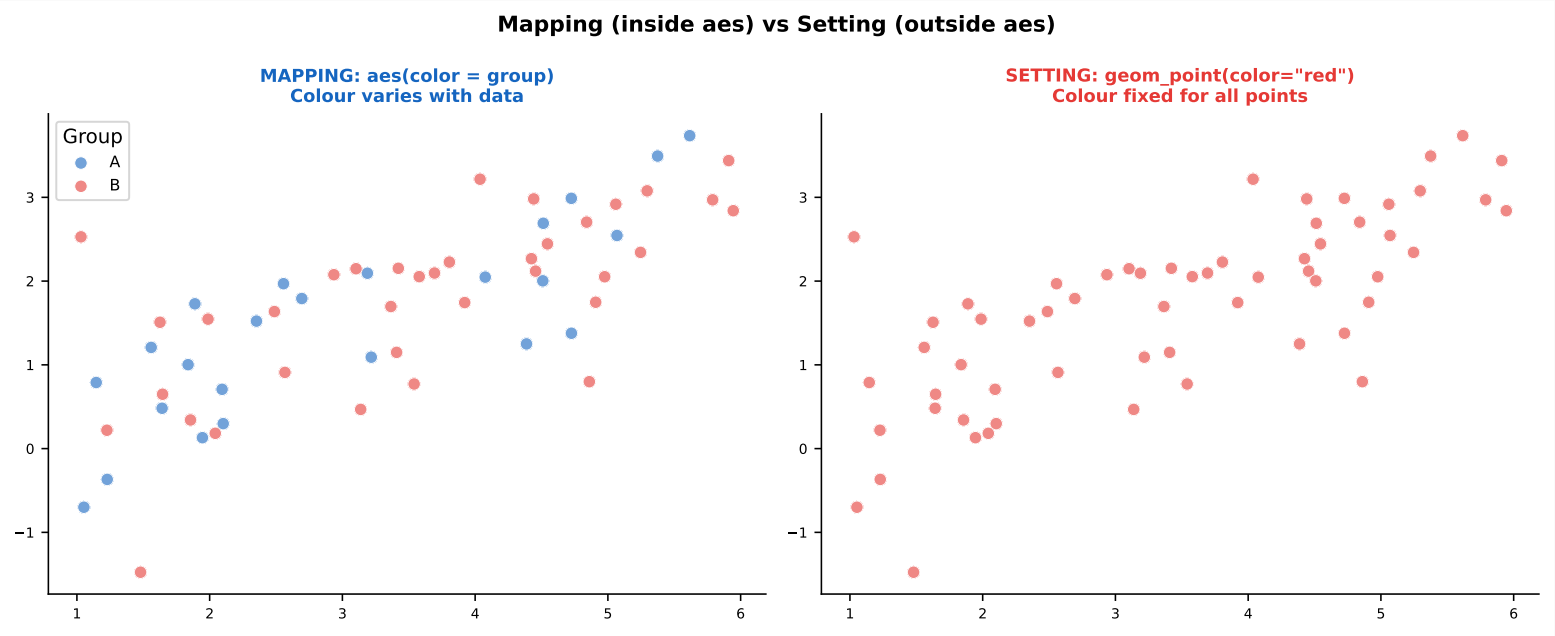
The + Operator Pipeline



Pro-Tip
The + must come at the *end* of each line in R. In plotnine, place + at the *start* inside parentheses.

Mapping vs Setting

The #1 ggplot2 Confusion



Mapping vs Setting in Code

MAPPING (data-driven)

```
# Colour varies with data
ggplot(df, aes(x = x, y = y,
    color = group)) +
  geom_point(size = 2)

# Size varies with data
ggplot(df, aes(x = x, y = y,
    size = population)) +
  geom_point(alpha = 0.6)
```

`aes()` creates a legend automatically.

SETTING (fixed)

```
# Colour fixed for all points
ggplot(df, aes(x = x, y = y)) +
  geom_point(color = "red",
             size = 3)

# Common mistake:
ggplot(df, aes(x = x, y = y,
    color = "red")) +
  geom_point()
# Maps string "red" as factor!
```

Fixed value inside `aes()` creates a useless single-level legend.

Mapping vs Setting in plotnine

MAPPING

```
from plotnine import *

(ggplot(df, aes(x="x", y="y",
    color="group"))
 + geom_point(size=2))
```

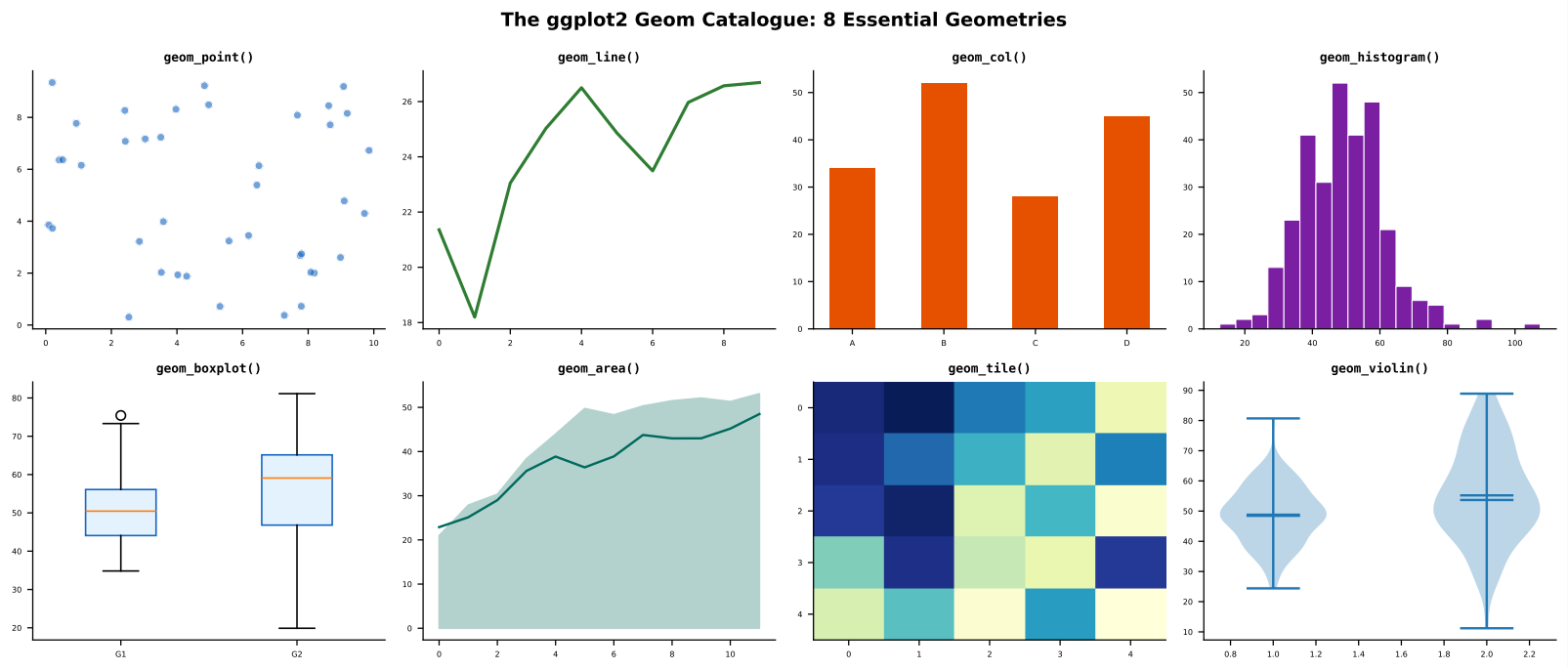
SETTING

```
from plotnine import *

(ggplot(df, aes(x="x", y="y"))
 + geom_point(color="red",
              size=3))
```

Data Insight
The rule is identical in R and Python: data-driven → inside `aes()`; fixed → outside `aes()`.

The Geom Catalogue

Eight Essential Geometries



Default Statistics per Geometry

Default Statistics per Geometry

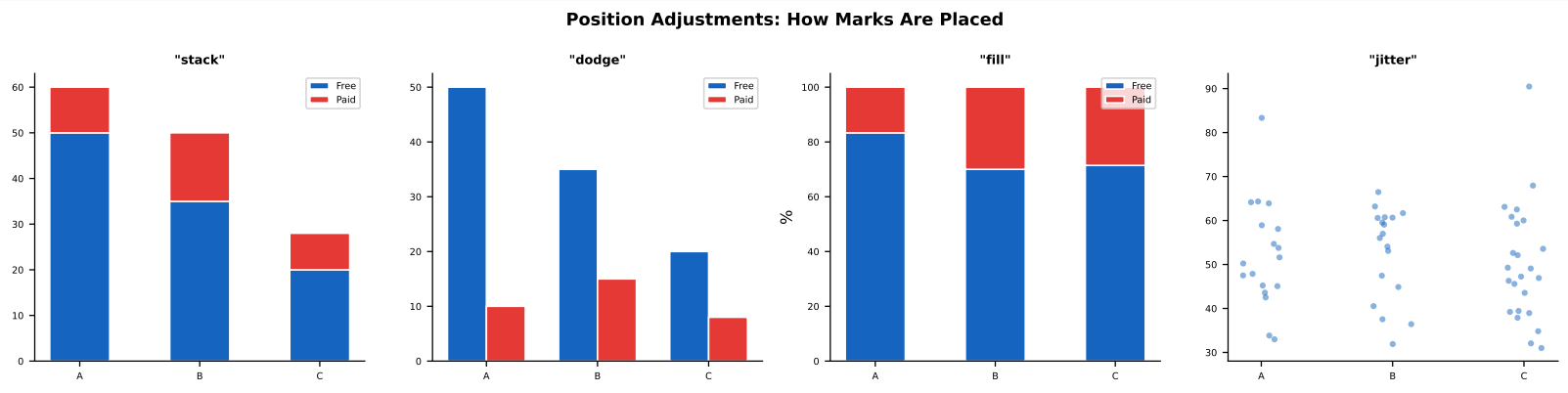
Geometry	Default Stat	What It Does
`geom_point()`	`stat_identity()`	Plots raw x,y values
`geom_line()`	`stat_identity()`	Connects raw points
`geom_col()`	`stat_identity()`	Bars from raw y heights
`geom_bar()`	`stat_count()`	Counts observations per x
`geom_histogram()`	`stat_bin()`	Bins continuous x, counts
`geom_boxplot()`	`stat_boxplot()`	Computes Q1,median,Q3,whiskers
`geom_smooth()`	`stat_smooth()`	Fits LOESS or lm + CI
`geom_density()`	`stat_density()`	Kernel density estimate

Pro-Tip

`geom_bar()` counts rows (`stat_count`); `geom_col()` plots raw heights (`stat_identity`). Confusing them is the #2 ggplot2 error.

Position Adjustments

Four Ways to Place Marks



Position Adjustments in R

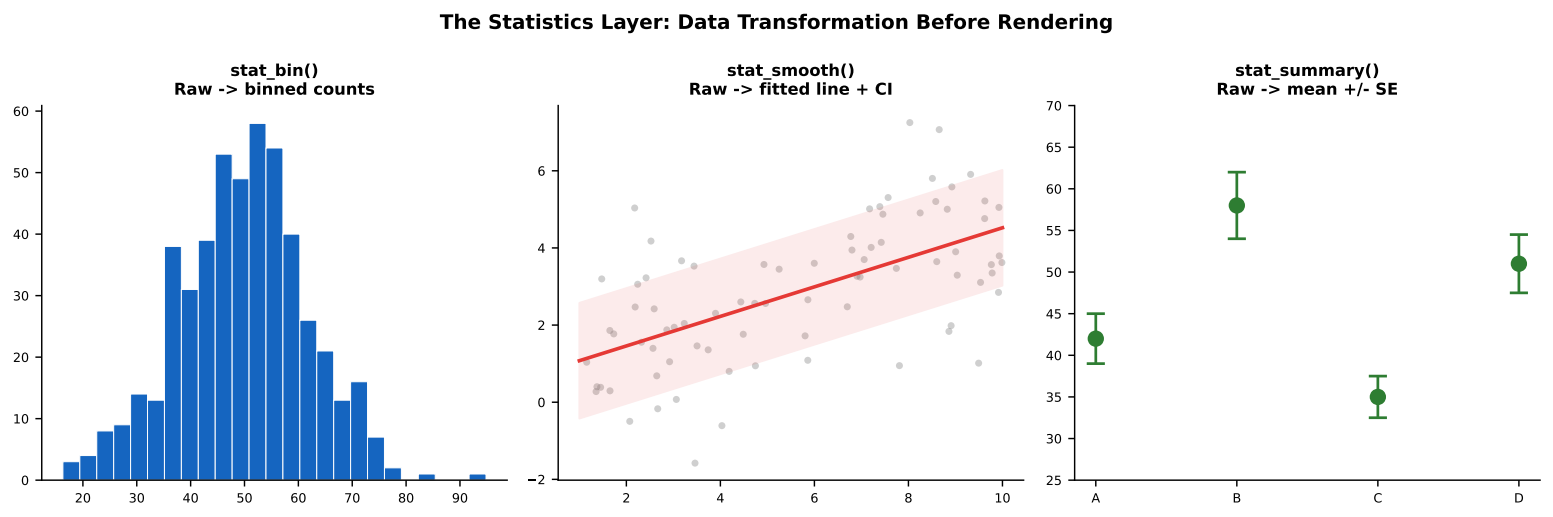
```r
# Stack (default for geom_bar with fill)
ggplot(df, aes(x = category, fill = type)) +
  geom_bar(position = "stack")

# Dodge (side by side)
ggplot(df, aes(x = category, fill = type)) +
  geom_bar(position = position_dodge(width = 0.8))

# Fill (proportional, sums to 100%)
ggplot(df, aes(x = category, fill = type)) +
  geom_bar(position = "fill") +
  scale_y_continuous(labels = scales::percent)

# Jitter (random noise to prevent overplotting)
ggplot(df, aes(x = category, y = rating)) +
  geom_jitter(width = 0.2, height = 0, alpha = 0.4, size = 1)
```

Position Adjustments in plotnine

```
from plotnine import *

# Stack
(ggplot(df, aes(x="category", fill="type")) + geom_bar(position="
    stack"))

# Dodge
(ggplot(df, aes(x="category", fill="type"))
 + geom_bar(position=position_dodge(width=0.8)))

# Fill
(ggplot(df, aes(x="category", fill="type")) + geom_bar(position="
    fill"))

# Jitter
(ggplot(df, aes(x="category", y="rating"))
 + geom_jitter(width=0.2, height=0, alpha=0.4, size=1))
```

The Statistics Layer

Statistics Transform Data Before Rendering



Data Insight

Every geom has a default stat, and every stat a default geom. You can override: `geom_bar(stat = "identity")` or `stat_smooth(geom = "point")`.

stat_summary: Compute on the Fly

R

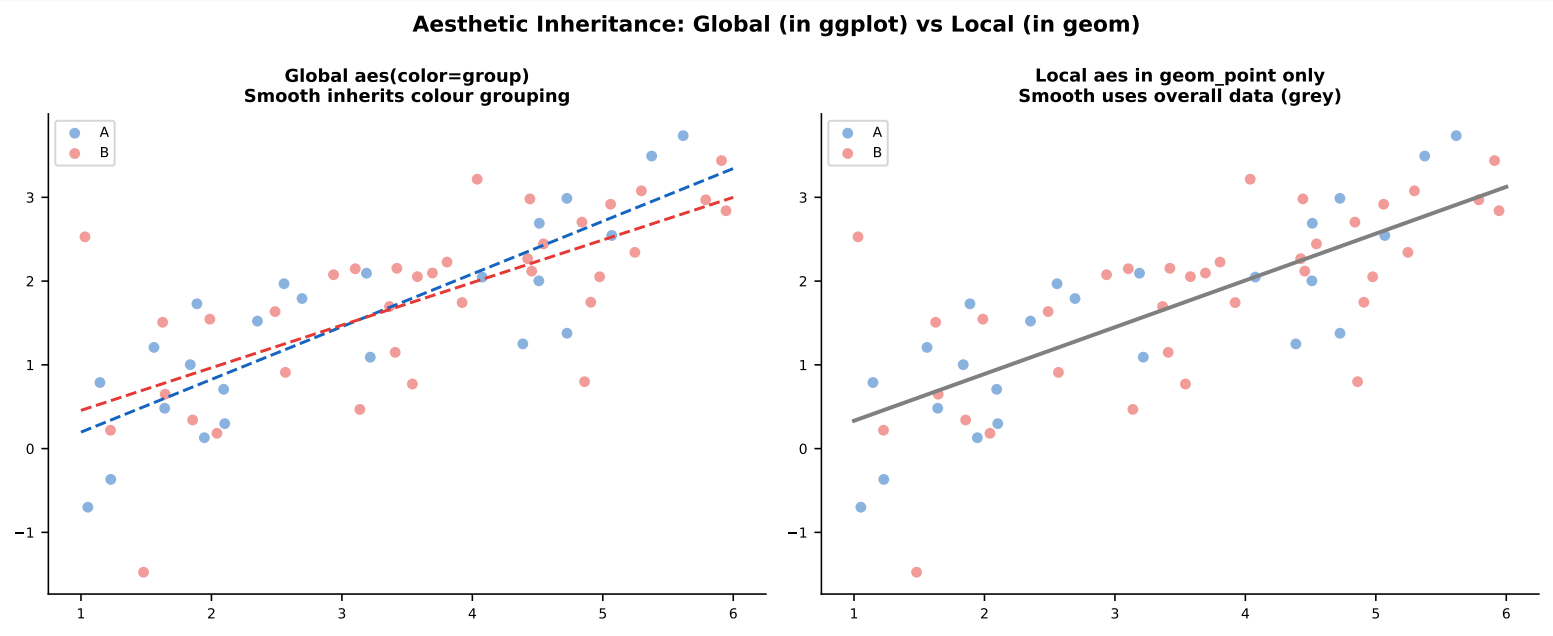
```r
ggplot(apps, aes(x = Category,
                 y = Rating)) +
  stat_summary(
    fun = mean,
    fun.min = function(x)
      mean(x)-sd(x)/sqrt(length(x
          )),
    fun.max = function(x)
      mean(x)+sd(x)/sqrt(length(x
          )),
    geom = "pointrange",
    color = "#E53935") +
  coord_flip() +
  theme_minimal()
```

Python (plotnine)

```python
(ggplot(apps, aes(
    x="Category", y="Rating"))
 + stat_summary(
     fun_y=np.mean,
     fun_ymin=lambda x:
       np.mean(x)-np.std(x)/
       np.sqrt(len(x)),
     fun_ymax=lambda x:
       np.mean(x)+np.std(x)/
       np.sqrt(len(x)),
     geom="pointrange",
     color="#E53935")
 + coord_flip()
 + theme_minimal())
```

Aesthetic Inheritance

Global vs Local Aesthetics



Inheritance Rules

Global (in `ggplot()`)

```
ggplot(df, aes(x = displ,
    y = hwy, color = class)) +
  geom_point() +
  geom_smooth()
# Both inherit color = class
# -> one smooth line per class
```

Local (in `geom_*()`)

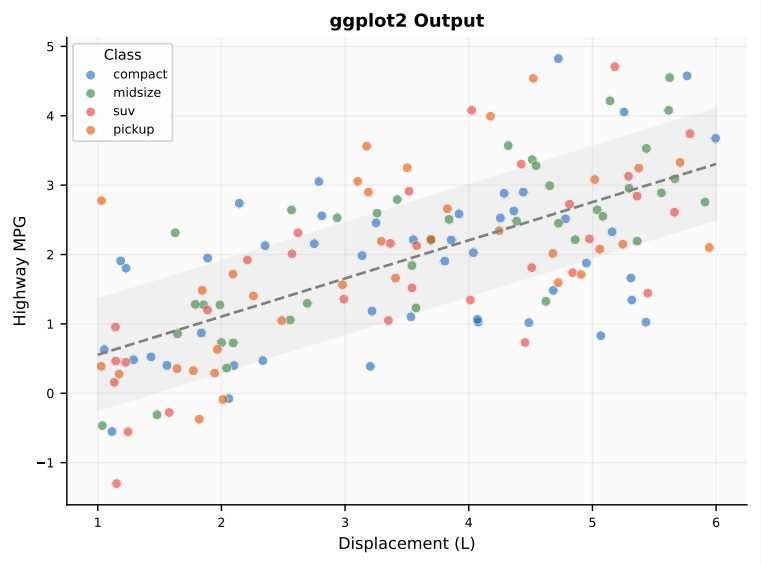
```
ggplot(df, aes(x = displ,
    y = hwy)) +
  geom_point(aes(color = class))
      +
  geom_smooth(color = "grey")
# Only points coloured by class
# Smooth = one overall grey line
```

Pro-Tip
Place shared aesthetics in `ggplot(aes())` and layer-specific aesthetics in `geom_*(aes())`. This controls which layers "see" a grouping variable.

Complete Worked Example

Complete Example: R

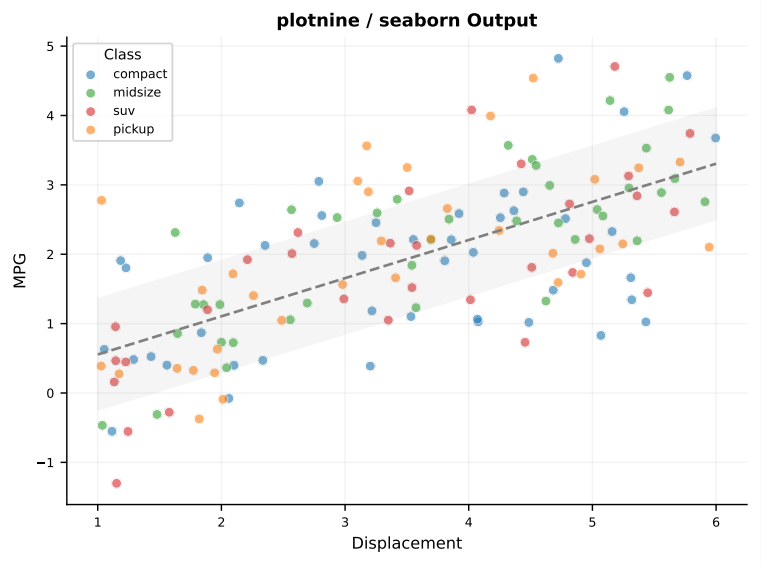
```r
ggplot(mpg,
  aes(x = displ, y = hwy,
      color = class)) +
  geom_point(size = 2, alpha = 0.6)
       +
  geom_smooth(
    aes(group = 1),
    method = "lm", se = TRUE,
    color = "grey40") +
  scale_color_brewer(
    palette = "Set2") +
  labs(
    title = "Displacement vs MPG",
    x = "Engine (L)",
    y = "Highway MPG",
    color = "Class") +
  theme_minimal(base_size = 11)
```



`aes(group=1)` overrides the global colour grouping in the smooth, producing one overall regression.

Complete Example: Python (plotnine)

```python
from plotnine import *

(ggplot(mpg,
    aes(x="displ", y="hwy",
        color="class"))
 + geom_point(size=2, alpha=0.6)
 + geom_smooth(
     aes(group=1),
     method="lm", se=True,
     color="grey", size=1)
 + scale_color_brewer(
     type="qual", palette="Set2")
 + labs(
     title="Displacement vs MPG",
     x="Engine (L)",
     y="Highway MPG",
     color="Class")
 + theme_minimal())
```



Near-identical code. Differences: strings for columns, + at line start, `size` vs `linewidth`.

Synthesis

Key Takeaways

1. Every ggplot2 chart follows the same **template**: `ggplot + geom + scale + facet + labs + theme`.
2. **Mapping** (inside `aes`) is data-driven; **setting** (outside `aes`) is fixed.
3. Each geom has a **default stat**: `geom_bar` counts; `geom_col` plots identity.
4. **Position adjustments** (dodge, stack, fill, jitter) control how marks overlap.
5. **Global** aesthetics are inherited by all layers; **local** apply to one layer only.
6. **plotnine** syntax is near-identical to ggplot2.

Essential Readings

- Wickham, H. (2010). "A Layered Grammar of Graphics." *JCGS*, 19(1), 3–28.
- Wickham, H. (2016). *ggplot2*, 2nd ed., Chapters 2–3.
- Wickham, H. & Grolemund, G. (2023). *R for Data Science*, 2nd ed., Chapters 9–12.
- plotnine documentation: `https://plotnine.org`

Next Module: ggplot2 Deep Dive — Aesthetics & Geometries (W02-M03)